Live Coding Python

Mini API, Client Résilient et Retry

Séance 6 — Communication, APIs et Fiabilité

Module : Applications Réparties
Pré-requis : Python, HTTP, JSON, notions auth (séance 5)

25 avril 2026

Table des matières

1 Introduction **3**

2 Exemple 1 — Mini API HTTP (`api_server.py`) **3**
2.1 Objectif pédagogique 3
2.2 Concepts abordés 3
2.3 Code source 3
2.4 Points pédagogiques 6
2.5 Résultat d'exécution 6

3 Exemple 2 — Client HTTP avec Timeout et Traçabilité (`api_client.py`) **7**
3.1 Objectif pédagogique 7
3.2 Concepts abordés 7
3.3 Code source 7
3.4 Points pédagogiques 9
3.5 Résultat d'exécution 9

4 Exemple 3 — Retry avec Backoff Exponentiel (`retry_backoff.py`) **10**
4.1 Objectif pédagogique 10
4.2 Concepts abordés 10
4.3 Formule du délai 10
4.4 Code source 10
4.5 Points pédagogiques 12
4.6 Résultat d'exécution 12

5 Synthèse Comparative **13**
5.1 Pièges fréquents à éviter 13

1 Introduction

Cette section présente trois extraits de code Python illustrant les concepts fondamentaux de la séance 6 : conception d'une mini API REST, client HTTP avec timeout et traçabilité, et retry avec backoff exponentiel.

N°	Fichier	Concept clé	Bibliothèque
1	`api_server.py`	Mini API REST (CRUD, Auth, Validation)	`http.server`
2	`api_client.py`	Client HTTP, Timeout, X-Request-Id	`urllib`
3	`retry_backoff.py`	Retry, Backoff exponentiel, Jitter	`time`, `random`

Table 1 – Vue d'ensemble des trois exemples

2 Exemple 1 — Mini API HTTP (`api_server.py`)

2.1 Objectif pédagogique

Implémenter une mini API REST exposant deux endpoints : `GET /health` (vérification de santé, sans auth) et `POST /documents` (création de document, avec authentification par token et validation d'entrée JSON). Cet exemple illustre les bonnes pratiques de conception d'API vues en cours.

2.2 Concepts abordés

- **Contrat d'API** : endpoints, méthodes HTTP, codes de retour.
- **Authentification par token** : vérification du header `Authorization: Bearer`.
- **Validation d'entrée** : vérification de la présence et des contraintes des champs JSON.
- **Codes HTTP appropriés** : 200, 201, 400, 401, 404.

2.3 Code source

```python
import json
import uuid
from http.server import HTTPServer, BaseHTTPRequestHandler
from datetime import datetime

# Stockage en memoire (simulation de base de donnees)
documents_db = {}

# Token attendu (en production : JAMAIS en dur !)
VALID_TOKEN = "secret-token-abc123"


class APIHandler(BaseHTTPRequestHandler):
    """Gestionnaire de requetes HTTP pour notre mini API."""

```

```python
    def _send_json(self, status_code, data):
        """Envoie une reponse JSON avec le code de statut donne."""
        self.send_response(status_code)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(data).encode("utf-8"))

    def _check_auth(self):
        """Verifie le header Authorization. Retourne True si valide."""
        auth = self.headers.get("Authorization", "")
        if auth != f"Bearer {VALID_TOKEN}":
            self._send_json(401, {
                "error": "unauthorized",
                "message": "Token manquant ou invalide"
            })
            return False
        return True

    def do_GET(self):
        if self.path == "/health":
            self._send_json(200, {
                "status": "ok",
                "timestamp": datetime.utcnow().isoformat(),
                "documents_count": len(documents_db)
            })
            return
        self._send_json(404, {
            "error": "not_found",
            "message": f"Endpoint {self.path} inconnu"
        })

    def do_POST(self):
        if self.path != "/documents":
            self._send_json(404, {
                "error": "not_found",
                "message": f"Endpoint {self.path} inconnu"
            })
            return

        # 1. Verifier l'authentification
        if not self._check_auth():
            return

        # 2. Lire et parser le body JSON
        content_length = int(
            self.headers.get("Content-Length", 0)
        )
        if content_length == 0:
            self._send_json(400, {
                "error": "bad_request",
                "message": "Corps de requete vide"
```

```
            })
            return

        try:
            raw_body = self.rfile.read(content_length)
            body = json.loads(raw_body)
        except json.JSONDecodeError:
            self._send_json(400, {
                "error": "bad_request",
                "message": "JSON invalide"
            })
            return

        # 3. Validation des champs
        title = body.get("title", "").strip()
        content = body.get("content", "").strip()

        if not title or not content:
            self._send_json(400, {
                "error": "validation_error",
                "message": "'title' et 'content' sont requis"
            })
            return

        if len(title) > 200:
            self._send_json(400, {
                "error": "validation_error",
                "message": "title ne doit pas depasser 200 car."
            })
            return

        # 4. Creer le document
        doc_id = str(uuid.uuid4())
        documents_db[doc_id] = {
            "id": doc_id,
            "title": title,
            "content": content,
            "created_at": datetime.utcnow().isoformat()
        }

        # 5. Repondre 201 Created
        self._send_json(201, documents_db[doc_id])

    def log_message(self, format, *args):
        pass


if __name__ == "__main__":
    server = HTTPServer(("127.0.0.1", 8080), APIHandler)
    print("Serveur demarre sur http://127.0.0.1:8080")
```

```
    server.serve_forever()
```

Listing 1 – api_server.py — Mini API REST avec Auth et Validation

2.4 Points pédagogiques

— La validation d'entrée est effectuée **avant** tout traitement métier : vérification du token (ligne auth), puis du body JSON, puis des champs.

— Chaque erreur retourne un code HTTP approprié : `401` pour l'auth, `400` pour la validation, `404` pour les routes inconnues.

— Les identifiants de documents sont des **UUID v4** (aléatoires) et non des entiers séquentiels, ce qui empêche l'énumération par un attaquant.

A retenir

En production, le token ne doit **jamais** être codé en dur. On utilise des variables d'environnement ou un vault sécurisé (HashiCorp Vault). Ce code est uniquement à visée pédagogique.

2.5 Résultat d'exécution

```
PS C:\Users\imada> cd Downloads
PS C:\Users\imada\Downloads> python api-server.py
==================================================
  Mini API REST — Seance 6
  Serveur demarre sur http://127.0.0.1:8080
  Endpoints : GET /health, POST /documents
==================================================
C:\Users\imada\Downloads\api-server.py:45: DeprecationWarning: datetime.datetime.utcnow() is d
eprecated and scheduled for removal in a future version. Use timezone-aware objects to represe
nt datetimes in UTC: datetime.datetime.now(datetime.UTC).
  "timestamp": datetime.utcnow().isoformat(),
127.0.0.1 - - [25/Apr/2026 14:51:34] "GET /health HTTP/1.1" 200 -
C:\Users\imada\Downloads\api-server.py:111: DeprecationWarning: datetime.datetime.utcnow() is
deprecated and scheduled for removal in a future version. Use timezone-aware objects to repres
ent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  "created_at": datetime.utcnow().isoformat()
127.0.0.1 - - [25/Apr/2026 14:51:34] "POST /documents HTTP/1.1" 201 -
127.0.0.1 - - [25/Apr/2026 14:51:34] "POST /documents HTTP/1.1" 401 -
127.0.0.1 - - [25/Apr/2026 14:51:34] "POST /documents HTTP/1.1" 400 -
127.0.0.1 - - [25/Apr/2026 14:51:34] "GET /inexistant HTTP/1.1" 404 -
127.0.0.1 - - [25/Apr/2026 14:51:44] "GET /health HTTP/1.1" 200 -
```

FIGURE 1 – Exécution de `api_server.py` : réponses JSON avec codes HTTP appropriés.

3 Exemple 2 — Client HTTP avec Timeout et Traçabilité (`api_client.py`)

3.1 Objectif pédagogique

Montrer comment construire un client HTTP qui intègre les bonnes pratiques de fiabilité (timeout) et d'observabilité (`X-Request-Id`) vues en cours.

3.2 Concepts abordés

— **Timeout** : durée maximale d'attente avant abandon.

— **X-Request-Id** : identifiant unique propagé pour la traçabilité (correlation ID).

— **Gestion des erreurs HTTP** : distinction entre erreurs HTTP (4xx/5xx) et erreurs réseau.

3.3 Code source

```python
import json
import uuid
from urllib.request import Request, urlopen
from urllib.error import HTTPError, URLError


def api_request(method, url, data=None, token=None,
                timeout=10):
    """
    Effectue une requete HTTP vers l'API.
    Retourne : (status_code, response_dict)
    """
    # Preparer le body JSON si necessaire
    body_bytes = None
    if data is not None:
        body_bytes = json.dumps(data).encode("utf-8")

    # Construire la requete
    req = Request(url, data=body_bytes, method=method)
    req.add_header("Content-Type", "application/json")

    # Ajouter un identifiant unique de requete
    request_id = str(uuid.uuid4())
    req.add_header("X-Request-Id", request_id)
    print(f"[{request_id[:8]}] {method} {url}")

    # Ajouter le token si fourni
    if token:
        req.add_header("Authorization",
                       f"Bearer {token}")

    # Envoyer avec timeout
```

```python
    try:
        with urlopen(req, timeout=timeout) as resp:
            body = json.loads(
                resp.read().decode("utf-8")
            )
            print(f"  -> {resp.status}")
            return resp.status, body

    except HTTPError as e:
        error_body = {}
        try:
            error_body = json.loads(
                e.read().decode("utf-8")
            )
        except Exception:
            pass
        print(f"  HTTP {e.code}: {error_body}")
        return e.code, error_body

    except URLError as e:
        print(f"  Erreur reseau: {e.reason}")
        return None, {"error": str(e.reason)}


if __name__ == "__main__":
    BASE = "http://127.0.0.1:8080"
    TOKEN = "secret-token-abc123"

    # GET /health (pas d'auth)
    status, data = api_request(
        "GET", f"{BASE}/health"
    )

    # POST /documents (avec auth + donnees)
    status, data = api_request(
        "POST", f"{BASE}/documents",
        data={"title": "Mon doc", "content": "Contenu"},
        token=TOKEN, timeout=5
    )

    # POST sans auth -> doit retourner 401
    status, data = api_request(
        "POST", f"{BASE}/documents",
        data={"title": "Test", "content": "X"}
    )
```

Listing 2 – api_client.py — Client HTTP avec timeout et X-Request-Id

3.4 Points pédagogiques

— Le `X-Request-Id` est un UUID unique généré par le client et envoyé via un header HTTP. Le serveur le loggue, permettant de **corréler** les logs client et serveur.

— La distinction entre `HTTPError` (erreur applicative 4xx/5xx) et `URLError` (erreur réseau) est fondamentale pour décider si un retry est pertinent.

— Le timeout de 10 secondes par défaut est un compromis raisonnable ; il est réduit à 5 secondes pour le POST.

A retenir

Le **correlation ID** (`X-Request-Id`) est indispensable en systèmes distribués : il permet de suivre une requête à travers tous les services qu'elle traverse pour le debugging.

3.5 Résultat d'exécution

```
PS C:\Users\imada> cd Downloads
PS C:\Users\imada\Downloads> python api.py
======================================================
  Client HTTP — Seance 6
  (Assurez-vous que api_server.py tourne d'abord)
======================================================

--- Test 1 : GET /health ---

[a4f56953] GET http://127.0.0.1:8080/health
  ✅ 200 — {'status': 'ok', 'timestamp': '2026-04-25T13:51:34.010854', 'documents_count': 0}

--- Test 2 : POST /documents (avec auth) ---

[89eb44be] POST http://127.0.0.1:8080/documents
  ✅ 201 — {'id': '24067af2-a8a5-4c85-a7d8-46c97da88a0f', 'title': 'Mon premier document', 'co
ntent': 'Contenu de test', 'created_at': '2026-04-25T13:51:34.013461'}

--- Test 3 : POST /documents (sans auth) ---

[c86f51f7] POST http://127.0.0.1:8080/documents
  ❌ HTTP 401: {'error': 'unauthorized', 'message': 'Token manquant ou invalide'}

--- Test 4 : POST /documents (body invalide) ---

[988dd167] POST http://127.0.0.1:8080/documents
  ❌ HTTP 400: {'error': 'validation_error', 'message': "'title' et 'content' sont requis"}

--- Test 5 : GET /inexistant ---

[ad990afb] GET http://127.0.0.1:8080/inexistant
  ❌ HTTP 404: {'error': 'not_found', 'message': 'Endpoint /inexistant inconnu'}

======================================================
```

FIGURE 2 – Exécution de `api_client.py` : trois requêtes avec codes de retour différenciés.

4 Exemple 3 — Retry avec Backoff Exponentiel (`retry_backoff.py`)

4.1 Objectif pédagogique

Implémenter le pattern **retry + backoff exponentiel + jitter** en intégrant les règles vues en cours : ne retenter que sur les erreurs transitoires (5xx, 429), jamais sur les erreurs client (4xx).

4.2 Concepts abordés

- **Backoff exponentiel** : le délai double à chaque échec (1s, 2s, 4s, 8s...).
- **Jitter (full jitter)** : `random.uniform(0, delay)` pour étaler les retries dans le temps.
- **Codes retryables vs non-retryables** : les erreurs 4xx ne sont jamais retentées (sauf 429).
- **Cas spécial 429** : respecter le `Retry-After`.

4.3 Formule du délai

Pour chaque tentative :

```
delay = min(base_delay * 2^attempt, max_delay)
jittered = random.uniform(0, delay)
```

Tentative	Délai de base	Délai max (plafonné)	Avec full jitter
0	1,0 s	1,0 s	0 – 1,0 s
1	2,0 s	2,0 s	0 – 2,0 s
2	4,0 s	4,0 s	0 – 4,0 s
3	8,0 s	8,0 s	0 – 8,0 s
4	16,0 s	16,0 s	0 – 16,0 s

Table 2 – Progression des délais avec `base_delay = 1.0` et `max_delay = 30.0`

4.4 Code source

```python
import time
import random


def request_with_retry(
    func,
    max_retries=4,
    base_delay=1.0,
    max_delay=30.0,
    retryable_statuses=(500, 502, 503, 504)
):
```

```python
    """
    Appelle func() avec retry + backoff exponentiel + jitter.
    func() doit retourner (status_code, response_body).
    Les erreurs 4xx ne sont JAMAIS retentees (sauf 429).
    """
    for attempt in range(max_retries + 1):

        status, body = func()

        # Succes -> retourner immediatement
        if status is not None and status < 500 \
                and status != 429:
            return status, body

        # Rate limited (429) -> respecter Retry-After
        if status == 429:
            wait = 30.0
            print(f"  Rate limited. Attente {wait}s")
            time.sleep(wait)
            continue

        # Derniere tentative -> abandonner
        if attempt == max_retries:
            print(
                f"  Abandon apres {max_retries+1} "
                f"tentatives"
            )
            return status, body

        # Calculer le delai de backoff exponentiel
        delay = min(
            base_delay * (2 ** attempt), max_delay
        )

        # Ajouter du jitter (full jitter)
        jittered_delay = random.uniform(0, delay)

        print(
            f"  Tentative {attempt+1}/{max_retries} "
            f"echouee (status={status}). "
            f"Retry dans {jittered_delay:.1f}s"
        )
        time.sleep(jittered_delay)

    return status, body


# Exemple d'utilisation
if __name__ == "__main__":

    def call_health():
```

```python
        return api_request(
            "GET",
            "http://127.0.0.1:8080/health",
            timeout=5
        )

    status, body = request_with_retry(call_health)
    print(f"Resultat final : {status} -> {body}")
```

Listing 3 – retry_backoff.py — Retry avec backoff exponentiel et jitter

4.5 Points pédagogiques

— Le **full jitter** (`random.uniform(0, delay)`) est recommandé par AWS : il étale les retries de manière optimale pour éviter les *retry storms*.

— Les erreurs `4xx` (sauf 429) ne sont **jamais** retentées : une requête invalide le restera.

— Le cas `429 Too Many Requests` est traité à part : en production, on lit le header `Retry-After`.

— Le code ne retente **pas** les POST non idempotents par défaut. Pour un POST avec `Idempotency-Key`, il faudrait ajouter cette logique.

A retenir

Ne jamais retenter aveuglément un `POST` sans mécanisme d'idempotence. Le retry est sûr pour les `GET` (lecture, idempotent), mais potentiellement dangereux pour les opérations d'écriture (double création, double paiement).

4.6 Résultat d'exécution

```
PS C:\Users\imada> cd Downloads
PS C:\Users\imada\Downloads> python retry.py
=====================================================
  Retry + Backoff Exponentiel — Seance 6
=====================================================

--- Test 1 : Retry vers GET /health ---
  ✅ Succes a la tentative 1 (status=200)
  Resultat final : status=200, body={'status': 'ok', 'timestamp': '2026-04-25T13:51:44.696659'
, 'documents_count': 1}

--- Test 2 : Retry vers un serveur inexistant ---
  🔄 Tentative 1/4 echouee (status=None). Retry dans 0.47s...
  🔄 Tentative 2/4 echouee (status=None). Retry dans 0.28s...
  🔄 Tentative 3/4 echouee (status=None). Retry dans 1.31s...
  💀 Abandon apres 4 tentatives
  Resultat final : status=None, body={'error': 'timed out'}

=====================================================
  Tests de retry termines.
=====================================================
PS C:\Users\imada\Downloads>
```

FIGURE 3 – Exécution de `retry_backoff.py` : retries avec backoff exponentiel et jitter visible.

5 Synthèse Comparative

Exemple	Pattern	Problème résolu	Quand l'utiliser
`api_server.py`	API REST, Auth, Validation	Exposer des ressources avec sécurité	Tout service HTTP
`api_client.py`	Timeout, Correlation ID	Blocage infini, traçabilité	**Tout** appel réseau
`retry_backoff.py`	Retry + backoff + jitter	Erreurs transitoires	Appels vers services critiques

5.1 Pièges fréquents à éviter

1. **Token en dur dans le code** → fuite de secret via Git. Utiliser des variables d'environnement.
2. **Retourner 200 avec un body d'erreur** → les clients automatisés se fient au code HTTP.
3. **Pas de validation d'entrée** → injections, données corrompues, crash serveur.
4. **Retry sur POST sans idempotency key** → duplication de ressources.
5. **Retry sans jitter** → retry storm (DDoS involontaire sur le service malade).